

Aim: Logistic Regression.**1.Importing the libraries**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import micropip
await micropip.install(['seaborn'])
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

2.Creating a random dataset with a twist.

```
np.random.seed(10)
data = {'Study_Hours' : np.random.normal(5,2,100).round(1),
        'Sleep_Hours' : np.random.normal(6,1.5,100).round(1)}
df = pd.DataFrame(data)
#1 = Pass AND 0 = Fail;
df['Passed'] = np.where((df['Study_Hours'] + df['Sleep_Hours']) > 10 ,1, 0)
#Flipping the 0 and 1;
flip_indices = np.random.choice(df.index, size = 10, replace = False)
df.loc[flip_indices, 'Passed'] = 1 - df.loc[flip_indices, 'Passed']
df.head()
```

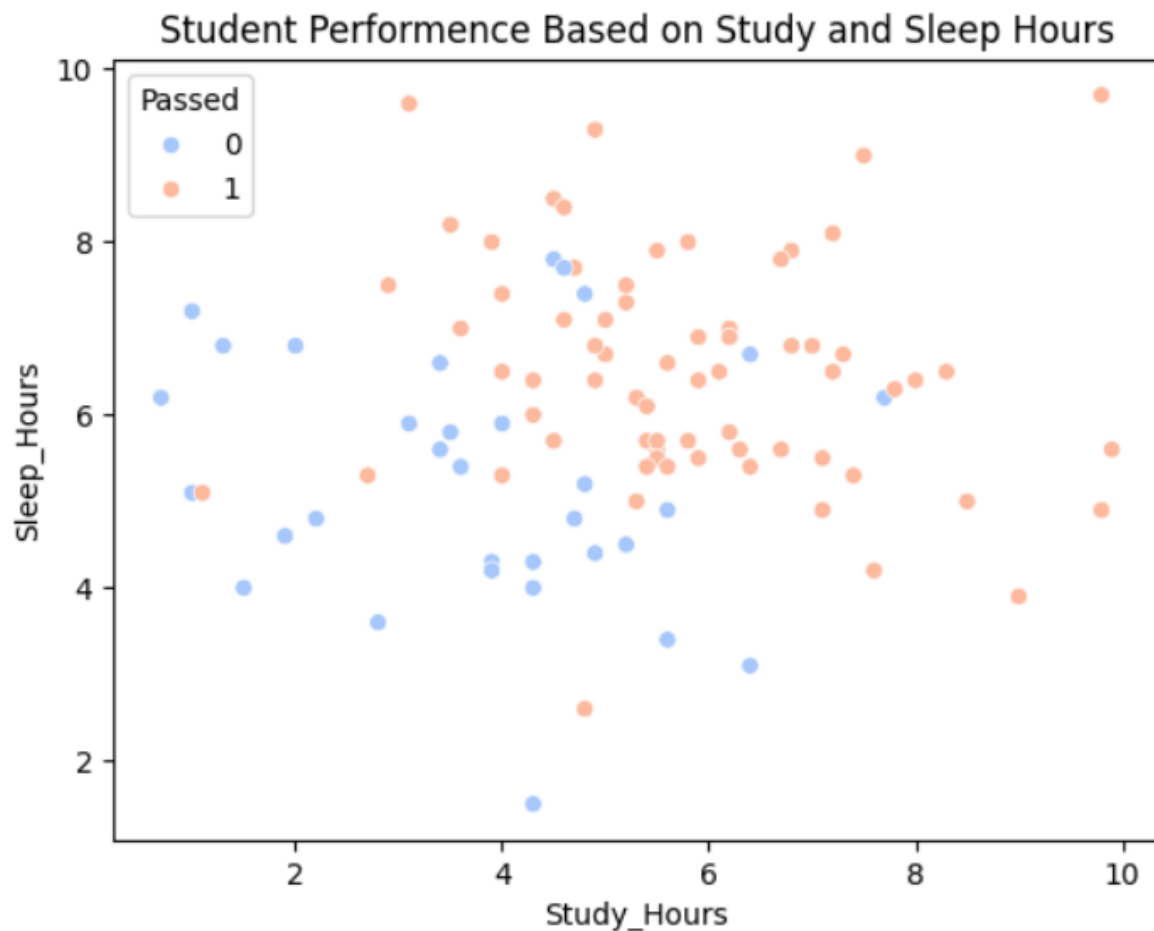
OUTPUT:

	Study_Hours	Sleep_Hours	Passed
0	7.7	6.2	0
1	6.4	3.1	0
2	1.9	4.6	0
3	5.0	6.7	1
4	6.2	5.8	1

3.Visualizing.

```
import seaborn as sns
sns.scatterplot(data=df, x='Study_Hours', y='Sleep_Hours', hue='Passed', palette='coolwarm')
plt.title("Student Performance Based on Study and Sleep Hours")
plt.show()
```

OUTPUT:



4.Training and testing the data.

```
x = df[['Study_Hours', 'Sleep_Hours']]
y = df['Passed']
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size = 0.2, random_state = 42)
```

5.Creating the model.

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

▼ LogisticRegression ⓘ ?

LogisticRegression()

6.Training the model.

```
y_pred = model.predict(X_test)
```

```
y_probs = model.predict_proba(X_test)[: , 1]
y_probs
```

OUTPUT:

```
array([0.82261867, 0.94438278, 0.45303979, 0.92038711, 0.53252178,
       0.79285252, 0.94713756, 0.95872287, 0.89868188, 0.9739423 ,
       0.98244519, 0.61367418, 0.9685866 , 0.64251763, 0.96749278,
       0.88331793, 0.27743176, 0.94252067, 0.87815037, 0.25976143])
```

7.Checking the accuracy, precision and classification report of the model.

```
print('Accuracy:', accuracy_score(y_test, y_pred))
print('Confusion Matrix\n', confusion_matrix(y_test, y_pred))
print('Classification Report\n', classification_report(y_test,y_pred))
```

OUTPUT:

Accuracy: 0.85

Confusion Matrix

```
[[ 3  3]
```

```
 [ 0 14]]
```

Classification Report

	precision	recall	f1-score	support
0	1.00	0.50	0.67	6
1	0.82	1.00	0.90	14
accuracy			0.85	20
macro avg	0.91	0.75	0.78	20
weighted avg	0.88	0.85	0.83	20